

**ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES  
BENI MELLAL**

Département Génie Logiciel

---

**RAPPORT DE TRAVAUX PRATIQUES**

TD Docker

*Application Multi-Conteneurs avec Docker Compose*

---

Encadré par : Pr. BE.ELBAGHAZAOUI  
Année universitaire 2025 - 2026

# Table des Matières

1. Introduction
2. Concepts fondamentaux de Docker
3. Étape 1 : Structure du projet
4. Étape 2 : Le Dockerfile
5. Étape 3 : Configuration de la base de données
6. Étape 4 : Docker Compose
7. Étape 5 : Tests et validation
8. Synthèse de l'architecture
9. Conclusion

## 1. Introduction

Ce rapport présente le travail réalisé dans le cadre du TD Docker du module Génie Logiciel, encadré par le Pr. BE.ELBAGHAZAOUI à l'ENSA de Beni Mellal. L'objectif de ce TP est de dockeriser une application web multi-conteneurs composée d'un serveur Nginx pour le frontend, d'une base de données PostgreSQL, et de l'interface d'administration pgAdmin.

Docker est une plateforme de conteneurisation qui permet d'emballer une application et toutes ses dépendances dans un conteneur léger, portable et reproductible. Docker Compose permet d'orchestrer plusieurs conteneurs qui forment ensemble une application complète.

Ce TP couvre la création d'un Dockerfile personnalisé, l'utilisation du fichier .dockerignore, la configuration des variables d'environnement, et l'orchestration via docker-compose.yml.

## 2. Concepts fondamentaux de Docker

| Concept               | Description                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Image</b>          | Modèle en lecture seule contenant le système de fichiers et la configuration nécessaire pour exécuter une application. Créée à partir d'un Dockerfile. |
| <b>Conteneur</b>      | Instance en cours d'exécution d'une image. Léger, isolé, et éphémère par défaut.                                                                       |
| <b>Dockerfile</b>     | Fichier texte contenant les instructions pour construire une image Docker personnalisée.                                                               |
| <b>Docker Compose</b> | Outil pour définir et exécuter des applications multi-conteneurs via un fichier YAML.                                                                  |
| <b>Volume</b>         | Mécanisme de persistance des données qui survit au cycle de vie du conteneur.                                                                          |
| <b>Network</b>        | Réseau virtuel permettant aux conteneurs de communiquer entre eux par nom de service.                                                                  |

## 3. Étape 1 : Structure du projet

### 3.1 Arborescence

```
project/
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

Le projet est organisé en deux dossiers principaux : frontend/ contenant le serveur web Nginx, et backend/ contenant la configuration de la base de données. Le fichier docker-compose.yml à la racine orchestre l'ensemble des services.

### 3.2 Fichier frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>Dockerized App</title>
</head>
<body>
  <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
```

## 4. Étape 2 : Le Dockerfile

### 4.1 Contenu du Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

### 4.2 Explication des instructions

| Instruction         | Rôle                                                                          |
|---------------------|-------------------------------------------------------------------------------|
| FROM nginx:alpine   | Utilise l'image officielle Nginx basée sur Alpine Linux (légère, ~5MB).       |
| COPY index.html ... | Copie le fichier index.html dans le répertoire de publication de Nginx.       |
| EXPOSE 80           | Déclare que le conteneur écoute sur le port 80 (documentation).               |
| CMD [...]           | Démarre Nginx en premier plan (daemon off) pour que le conteneur reste actif. |

## 4.3 Fichier .dockerignore

```
*.log
```

Le fichier `.dockerignore` fonctionne comme `.gitignore` : il exclut les fichiers correspondant aux motifs spécifiés du contexte de build Docker. Ici, les fichiers de log sont exclus pour réduire la taille de l'image et accélérer le build.

## 5. Étape 3 : Configuration de la base de données

### 5.1 Fichier backend/db.env

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

Ce fichier contient les variables d'environnement utilisées par l'image officielle PostgreSQL pour initialiser automatiquement la base de données lors du premier démarrage. L'utilisation d'un fichier .env séparé permet de centraliser la configuration et de la séparer du code source.

### 5.2 Variables expliquées

| Variable          | Description                                              |
|-------------------|----------------------------------------------------------|
| POSTGRES_USER     | Nom de l'utilisateur superadmin de PostgreSQL (admin)    |
| POSTGRES_PASSWORD | Mot de passe de l'utilisateur PostgreSQL (adminpassword) |
| POSTGRES_DB       | Nom de la base de données créée automatiquement (myapp)  |

## 6. Étape 4 : Docker Compose

### 6.1 Fichier docker-compose.yml complet

```
version: "3.8"
services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "8080:80"
    networks:
      - app-network

  db:
    image: postgres:13
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    env_file:
      - ./backend/db.env
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network

  pgadmin:
    image: dpage/pgadmin4
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@admin.com
```

```
    - PGADMIN_DEFAULT_PASSWORD=admin
  ports:
    - "5050:80"
  depends_on:
    - db
  networks:
    - app-network

volumes:
  db-data:

networks:
  app-network:
```

## 6.2 Analyse des services

| Service  | Image                       | Port           | Volume  | Rôle                                 |
|----------|-----------------------------|----------------|---------|--------------------------------------|
| frontend | nginx:alpine (custom build) | 8080:80        | —       | Serveur web affichant la page HTML   |
| db       | postgres:13                 | 5432 (interne) | db-data | Base de données PostgreSQL           |
| pgadmin  | dpage/pgadmin4              | 5050:80        | —       | Interface d'administration de la BDD |

## 6.3 Éléments clés du fichier

**Réseau (app-network) :** Tous les services partagent le même réseau Docker, ce qui leur permet de communiquer entre eux par nom de service (ex: le host 'db' est résolu automatiquement vers le conteneur PostgreSQL).

**Volume (db-data) :** Le volume nommé db-data est monté sur `/var/lib/postgresql/data` dans le conteneur db. Il assure la persistance des données : même si le conteneur est supprimé et recréé, les données de la base sont conservées.

**depends\_on :** La directive `depends_on` dans le service pgadmin garantit que le conteneur de base de données (db) est démarré avant pgAdmin. Cela assure que pgAdmin peut se connecter à PostgreSQL dès son démarrage.



## 7. Étape 5 : Tests et validation

### 7.1 Lancement des conteneurs

```
$ docker-compose up --build
```

Cette commande construit l'image du frontend à partir du Dockerfile, télécharge les images postgres:13 et dpage/pgadmin4 depuis Docker Hub, crée le réseau et le volume, puis démarre les trois conteneurs.

### 7.2 Vérification du frontend

**URL :** http://localhost:8080

Le navigateur affiche la page "Bienvenue dans l'application Dockerisée", confirmant que Nginx sert correctement le fichier index.html depuis le conteneur.

### 7.3 Connexion à pgAdmin

**URL :** http://localhost:5050

| Paramètre          | Valeur          |
|--------------------|-----------------|
| Email de connexion | admin@admin.com |
| Mot de passe       | admin           |

### 7.4 Ajout d'une connexion PostgreSQL dans pgAdmin

| Paramètre | Valeur                     |
|-----------|----------------------------|
| Host      | db (nom du service Docker) |
| Port      | 5432                       |
| Username  | admin                      |
| Password  | adminpassword              |

Le host est 'db' et non 'localhost' car les conteneurs communiquent via le réseau Docker interne (app-network). Docker résout automatiquement le nom du service vers l'adresse IP du conteneur correspondant.

### 7.5 Test de persistance

```
$ docker-compose down
$ docker-compose up
# Les données sont conservées grâce au volume db-data

$ docker volume ls
# Affiche le volume project_db-data
```

## 8. Synthèse de l'architecture

Le schéma suivant résume l'architecture de l'application dockerisée :

| Frontend (Nginx)              | Base de données (PostgreSQL)  | Admin (pgAdmin4)       |
|-------------------------------|-------------------------------|------------------------|
| Port externe : 8080           | Port interne : 5432           | Port externe : 5050    |
| Image : nginx:alpine (custom) | Image : postgres:13           | Image : dpage/pgadmin4 |
| Sert index.html               | Volume : db-data (persistant) | depends_on : db        |

Les trois services sont connectés via le réseau app-network, permettant une communication transparente par nom de service.

## 9. Conclusion

Ce TP nous a permis de découvrir et de mettre en pratique les concepts fondamentaux de Docker et Docker Compose. Nous avons appris à créer un Dockerfile personnalisé pour servir une page web avec Nginx, à configurer une base de données PostgreSQL avec des variables d'environnement, et à orchestrer l'ensemble avec Docker Compose.

Les points clés retenus sont l'importance des volumes pour la persistance des données, l'utilisation des réseaux Docker pour la communication inter-conteneurs, et la puissance de Docker Compose pour déployer une application multi-services en une seule commande. Ces compétences sont directement applicables dans un environnement professionnel pour le déploiement d'applications en production.